Kraków bez barier: instrukcja uruchomienia

Zespół Pioneer Collective · Demo: **https://jakub.byst.re** · Kod: github.com/LucasBoj/krakow-bez-barier · Stan: 3.11.2026

1. Demo online (bez instalacji)

Aplikacja działa pod adresem **https://jakub.byst.re** w każdej nowoczesnej przeglądarce, na komputerze i telefonie. Nie wymaga konta.

Ścieżka na 6 minut

1. **Strona główna → „Pokaż przykład”.** Aplikacja ustawia profil „Wózek ręczny” i otwiera kartę Urzędu Miasta Krakowa: status, powody oceny, źródło i data każdej cechy, najbliższe koperty ZDMK. Cechy oznaczone „dane demonstracyjne” są danymi zespołu.
2. **„Profil potrzeb”**: wybierz inny profil (np. „Spektrum autyzmu” albo „Senior”) i wróć do tej samej karty: ocena i powody się zmieniają.
3. **„Miejsca” → „Dodaj do planu”** przy 3–4 miejscach, potem **„Plan”**: kolejność, trasa po ulicach, ostrzeżenia o hałasie na odcinkach, ławki i toalety na przerwy, przejazd tramwajem lub autobusem według rozkładu, „Czytaj na głos”.
4. **„Luki w danych i bariery”** (`/miasto` , link na stronie głównej i w stopce): raport dla urzędu z danymi o dzielnicach, udogodnieniami w zasięgu mieszkańców i seniorów (wykres korelacji), listą 20 miejsc do sprawdzenia i eksportem CSV/GeoJSON.
5. **„Planer inwestycji”** (`/planer-inwestycji` , przycisk na stronie raportu): które przystanki doposażyć w wiaty i które toalety dostosować najpierw, ilu mieszkańców i seniorów zyska dostęp; suwak zmienia liczbę inwestycji.
6. **„Panel urzędnika”** (`/urzednik`): dane logowania konta demonstracyjnego podajemy w treści maila zgłoszeniowego. W panelu widać zgłoszenia mieszkańców, potwierdzanie braków i barier oraz zakładkę „Braki danych”.

Inne strony: „Źródła danych” (`/zrodla-danych` : każdy zbiór z adresem, licencją i datą), „O projekcie” (`/o-projekcie`), opis API w OpenAPI (`/openapi/v1.json`). Przycisk „Samouczek” w nagłówku prowadzi po aplikacji krok po kroku.

2. Uruchomienie lokalne

Wymagania: .NET SDK 10 (`https://dotnet.microsoft.com/download/dotnet/10.0`), Git. System: Windows, macOS albo Linux.

```
git clone https://github.com/LucasBoj/krakow-bez-barier.git
cd krakow-bez-barier
dotnet run --project src/Web --launch-profile http
```

Aplikacja działa pod adresem `http://localhost:5008` . Dane miejsc, rozkład i siatki są w repozytorium, więc nic nie trzeba importować.

Bez żadnych kluczy działa: przeglądanie i ocena miejsc, profil, plan dnia (trasy w linii prostej), raport `/miasto` , źródła danych. Zgłoszenia, konta i panel urzędnika wymagają bazy MongoDB; bez niej pokazują komunikat „baza niedostępna”.

Opcjonalna konfiguracja (sekrety lokalnie)

```
dotnet user-secrets set "Mongo:ConnectionString" "mongodb+srv://<użytkownik>:<hasło>@<klaster>/" --project src/Web
dotnet user-secrets set "OpenRouteService:ApiKey" "<klucz>" --project src/Web
dotnet user-secrets set "OpenAI:ApiKey" "<klucz>" --project src/Web
dotnet user-secrets set "Officials:Seed:0:Login" "<login urzędnika>" --project src/Web
dotnet user-secrets set "Officials:Seed:0:Password" "<hasło urzędnika>" --project src/Web
```

Ustawienie	Co włącza	Bez niego
`Mongo:ConnectionString`	konta, plany na koncie, zgłoszenia, panel urzędnika (wystarczy darmowy klaster MongoDB Atlas M0)	komunikat „baza niedostępna”
`OpenRouteService:ApiKey`	trasy po ulicach dobrane do profilu (darmowy klucz na openrouteservice.org)	odcinki w linii prostej
`OpenAI:ApiKey`	ocena zdjęcia przeszkody w zgłoszeniu	zdjęcie trafia do zgłoszenia bez oceny
`Officials:Seed:0:*`	konto urzędnika zakładane przy starcie	brak dostępu do panelu urzędnika

3. Uruchomienie w kontenerze (jak na serwerze)

```
docker build -t krakow-bez-barier .
docker run -p 8080:8080 \
  -e Mongo__ConnectionString="mongodb+srv://..." \
  -e OpenRouteService__ApiKey="..." \
  krakow-bez-barier
```

Aplikacja działa pod adresem `http://localhost:8080`. Zmienne środowiskowe mają te same nazwy co ustawienia, z `__` zamiast `:`. Wdrożenie produkcyjne (GitHub Actions → VPS) opisuje README, sekcja „Wdrożenie na Mikrusa”.

4. Testy, audyt dostępności, odświeżenie danych

```
dotnet test CracowWithoutBarriers.slnx                # 304 testy jednostkowe

cd tools/a11y && npm ci && npx playwright install chromium
npm run audit                                          # axe-core, WCAG 2.2 A/AA (aplikacja musi działać)

dotnet run --project src/Tools -- city krakow          # odświeżenie danych miasta z MSIP i ZDMK
dotnet run --project src/Tools -- report krakow        # liczby raportu „Luki w danych” do doc/
```

Import pobiera dane z publicznych usług miasta (ArcGIS REST MSIP), więc wymaga internetu i trwa kilka minut. Każdy zbiór trafia do spisu źródeł z datą pobrania (`doc/DATA_SOURCES.md`).